

Lab Week 3: Automated ETL with API, SQLite and Dashboard

การสร้าง Automated Multi-source ETL Pipeline ด้วย KNIME

คำแนะนำ

ให้ปฏิบัติตามขั้นตอนใน KNIME ตามลำดับ โดยทุกขั้นตอนต้องตรวจสอบจำนวนแถวและสิ่งที่เกิดขึ้นกับข้อมูล ห้ามแก้ไขข้อมูลเป็นรายแถวด้วยตนเอง แต่ให้กำหนดเงื่อนไขหรือกฎที่ระบบสามารถนำไปใช้ซ้ำกับข้อมูลชุดใหม่ได้

แนวคิดสำคัญ

Week 3 เน้น Rule-driven ETL คือการกำหนดกฎครั้งเดียว แล้วให้ Workflow นำกฎนั้นไปใช้กับข้อมูลใหม่โดยอัตโนมัติ รวมถึงการดึงข้อมูลจาก API การแยก Reject Data การจัดเก็บข้อมูลลง SQLite และการเตรียมข้อมูลสำหรับ Dashboard

1. วัตถุประสงค์การเรียนรู้

- สร้าง ETL Pipeline จากหลายแหล่งข้อมูลได้
- กำหนด Data Quality Rule ให้ระบบตรวจข้อมูลอัตโนมัติได้
- เชื่อม Transaction Data กับ Master Data ได้
- เรียก REST API และแปลง JSON เป็น Table ได้
- เชื่อมข้อมูล Sales กับ Weather ด้วย Composite Key ได้
- แยก Valid Data และ Reject Data ได้
- Load ข้อมูลลง SQLite Database ได้
- เตรียมข้อมูลสำหรับสร้าง Dashboard ใน Power BI ได้

2. ข้อมูลที่ใช้

- Sales_Transactions.csv
Fields: Sales_ID, Date, Branch_ID, Product_ID, Quantity, Discount_Amount
- Product_Master.csv
Fields: Product_ID, Product_Name, Category, Unit_Price
- Branch_Master.csv
Fields: Branch_ID, Branch_Name, Latitude, Longitude
- Open-Meteo Historical Weather API
Fields ที่ใช้: time, temperature_2m_mean, temperature_2m_max, precipitation_sum, weather_code
ช่วงวันที่: 2026-09-01 ถึง 2026-09-15

ส่วนที่ 1 — สาย Sales: อ่านและตรวจคุณภาพข้อมูลขาย

เริ่มสายใหม่จาก `Sales_Transactions.csv`

ขั้นตอนที่ 1: อ่านข้อมูล Sales

Node: CSV Reader

รับข้อมูลจาก: `Sales_Transactions.csv`

ต่อไปยัง: Duplicate Row Filter

วิธีตั้งค่า

- ลาก CSV Reader ลง Workflow
- Double-click ที่ Node
- กด Browse
- เลือกไฟล์ `Sales_Transactions.csv`
- ตรวจสอบว่า Delimiter เป็น comma
- ตรวจสอบว่าใช้แถวแรกเป็นชื่อ Column
- ตรวจสอบว่ามี 6 Columns ได้แก่ Sales_ID, Date, Branch_ID, Product_ID, Quantity, Discount_Amount
- กด Apply and Execute

วัตถุประสงค์ของขั้นตอนนี้

เป็นขั้นตอน Extract เพื่อนำข้อมูลรายการขายเข้าสู่ KNIME โดยยังไม่แก้ไขข้อมูลต้นฉบับ

ผลที่ควรได้

46 rows

บันทึกผล

จำนวนแถว 46

จำนวนคอลัมน์ 6

ขั้นตอนที่ 2: ตรวจข้อมูลซ้ำ

Node: Duplicate Row Filter

รับข้อมูลจาก: CSV Reader – Sales

ต่อไปยัง: Missing Value

วิธีตั้งค่า

- Double-click Duplicate Row Filter
- เลือก Sales_ID เป็น Column สำหรับตรวจ Duplicate
- กำหนดให้เก็บรายการแรก
- Execute

วัตถุประสงค์ของขั้นตอนนี้

หนึ่ง Sales_ID ควรแทนรายการขายหนึ่ง Transaction หาก Sales_ID ซ้ำ ยอดขายอาจถูกนับซ้ำ

ผลที่ควรได้

ก่อน = 46 rows

หลัง = 45 rows

ถูกตัด = 1 row

คำถามสะท้อนคิด

เหตุใดจึงใช้ Sales_ID แทน Product_ID ตรวจ Duplicate?

ตอบ เพราะว่า Sales_ID เป็น primary key ของ Sales_Transactions.csv ซึ่งมันไม่ควรซ้ำกัน

ขั้นตอนที่ 3: จัดการ Discount ที่หายไป

Node: Missing Value

รับข้อมูลจาก: Duplicate Row Filter

ต่อไปยัง: Rule-based Row Splitter – Quantity

วิธีตั้งค่า

1. Double-click Missing Value
2. เลือก Column Discount_Amount
3. กำหนด Missing numeric value เป็น 0
4. Execute

วัตถุประสงค์ของขั้นตอนนี้

ในเจตยานี้กำหนด Business Rule ว่า หากไม่มีการระบุส่วนลด ให้ถือว่าไม่มีส่วนลด ดังนั้น Missing Discount → 0

หมายเหตุ: ไม่ได้หมายความว่า Missing ทุกประเภทควรแทนด้วย 0

ผลที่ควรได้

45 rows

บันทึกผล

Missing ก่อน 45

Missing หลัง 45

ขั้นตอนที่ 4: ตรวจสอบ Quantity

Node: Rule-based Row Splitter

รับข้อมูลจาก: Missing Value

ต่อไปยัง:

- TRUE Output → ขั้นตอนที่ 6 Join Product
- FALSE Output → พักไว้ทำ Reject Log ในส่วนที่ 7

วิธีตั้งค่า

ใส่ Rule:

```
MISSING $Quantity$ => FALSE
$Quantity$ <= 0 => FALSE
TRUE => TRUE
```

วัตถุประสงค์ของขั้นตอนนี้

เราไม่ควรกำหนดว่า Sales_ID ใดผิด แต่ควรกำหนดลักษณะของข้อมูลผิด เช่น Quantity <= 0 เพื่อให้ Workflow เดิมใช้กับไฟล์ใหม่ได้โดยอัตโนมัติ

ผลที่ควรได้

Valid = 44 rows

Reject = 1 row

บันทึกผล

Valid 44

Reject 1

คำถามสะท้อนคิด

เหตุใดการกำหนด Rule จากเงื่อนไขข้อมูลจึงดีกว่าการระบุ Sales_ID ที่ผิด?

ตอบ เพราะสามารถตรวจจับข้อมูลผิดพลาดชุดใหม่ที่ไม่เคยเห็นมาก่อนได้

ส่วนที่ 2 — สาย Product Master

เริ่มสายใหม่จาก *Product_Master.csv*

ขั้นตอนที่ 5: อ่าน Product Master

Node: CSV Reader

รับข้อมูลจาก: Product_Master.csv

ต่อไปยัง: Joiner – Product

วิธีตั้งค่า

1. เพิ่ม CSV Reader ตัวใหม่
2. เลือก Product_Master.csv
3. ตรวจสอบ Columns ได้แก่ Product_ID, Product_Name, Category, Unit_Price
4. Execute

วัตถุประสงค์ของขั้นตอนนี้

Sales มีเพียง Product_ID แต่ยังไม่มียี่ห้อสินค้า ประเภทสินค้า และราคา จึงต้องนำ Master Data มาเชื่อม

ผลที่ควรได้

8 rows

บันทึกผล

จำนวนสินค้า 8 รายการ

ส่วนที่ 3 — รวม Sales กับ Product และเตรียม Sales ให้พร้อม

นำสาย Sales และ Product Master มารวมกัน

ขั้นตอนที่ 6: Join Sales กับ Product Master

Node: Joiner

รับข้อมูลจาก

Top / Left input: Valid Output จาก Quantity Splitter = 44 rows

Bottom / Right input: Product Master = 8 rows

วิธีตั้งค่า Matching Criteria

Top Product_ID = Bottom Product_ID

Row Selection

เลือก:

- ✓ Matching rows
- ✓ Left unmatched rows

Output Columns

ดึง Sales เก็บ Columns เดิมทั้งหมด

ดึง Product Master เลือกเฉพาะ:

Product_Name

Category

Unit_Price

ไม่ต้องเลือก Product_ID ดึงซ้ำ

วัตถุประสงค์ของขั้นตอนนี้

Matching rows คือ Product ที่พบใน Master

Left unmatched rows ต้องเก็บไว้ เพราะเราต้องการตรวจ Product_ID ที่ไม่มีใน Master ไม่ใช่ว่าปล่อยให้ข้อมูลนั้นหายไป

ผลที่ควรได้

44 rows

บันทึกผล

จำนวนก่อน Join 44

จำนวนหลัง Join 44

จำนวนที่ Join ไม่สำเร็จ 1

ขั้นตอนที่ 7: แยก Product ที่ไม่พบใน Master

Node: Rule-based Row Splitter

รับข้อมูลจาก: Joiner – Product

ต่อไปยัง:

- TRUE Output → Math Formula – Gross Sales
- FALSE Output → พักไว้ทำ Reject Log ในส่วนที่ 8

วิธีตั้งค่า

```
MISSING $Product_Name$ => FALSE  
TRUE => TRUE
```

วัตถุประสงค์ของขั้นตอนนี้

ถ้า Product_ID ไม่มีใน Product Master การ Join จะไม่สามารถเติม Product_Name ได้ ดังนั้น Product_Name = Missing ใช้เป็นเงื่อนไขตรวจ Product ที่ไม่ถูกต้อง

ผลที่ควรได้

Valid = 43 rows

Reject = 1 row

ขั้นตอนที่ 8: คำนวณ Gross Sales

Node: Math Formula

รับข้อมูลจาก: TRUE Output ของ Product Splitter

ต่อไปยัง: Math Formula – Net Sales

วิธีตั้งค่า

สร้าง Column ใหม่ชื่อ Gross_Sales

Formula:

```
$Unit_Price$ * $Quantity$
```

วัตถุประสงค์ของขั้นตอนนี้

Gross Sales คือมูลค่าการขายก่อนหัก Discount

ผลที่ควรได้

43 rows

ตัวอย่าง

Unit_Price _____ Quantity _____ Gross_Sales _____

Unit_Price _____ Quantity _____ Gross_Sales _____

Unit_Price _____ Quantity _____ Gross_Sales _____

ขั้นตอนที่ 9: คำนวณ Net Sales

Node: Math Formula

รับข้อมูลจาก: Gross Sales

ต่อไปยัง: String to Date&Time

วิธีตั้งค่า

สร้าง Column ใหม่ชื่อ Net_Sales

Formula:

```
$Gross_Sales$ - $Discount_Amount$
```

วัตถุประสงค์ของขั้นตอนนี้

Net_Sales คือยอดขายสุทธิที่จะนำไปใช้เป็น Metric หลักใน Dashboard

ผลที่ควรได้

43 rows

คำถามสะท้อนคิด

Gross Sales และ Net Sales แตกต่างกันอย่างไร?

ตอบ ต่างกันตรงที่ Gross Sales ยอดขายรวมทั้งหมดก่อนหักส่วนลด แต่ Net Sales คือ ยอดขายจริงหลังจากหักส่วนลดทางการค้าเรียบร้อยแล้ว

ขั้นตอนที่ 10: แปลงวันที่ Sales

Node: String to Date&Time

รับข้อมูลจาก: Net Sales

ต่อไปยัง: พักไว้เป็น SALES READY

วิธีตั้งค่า

1. ใน Includes เลือกเฉพาะ Date
2. Output type = Date
3. ถ้าค่าเป็น 2026-09-01 ใช้ Format = yyyy-MM-dd
4. ถ้าไฟล์จาก Excel แสดงเป็น 1/9/2026 ใช้ Format = d/M/yyyy
5. Output columns = Replace
6. Execute

วัตถุประสงค์ของขั้นตอนนี้

Sales ต้อง Join กับ Weather ด้วย Date ดังนั้น Date ของทั้งสองฝั่งต้องมี Data Type เดียวกัน

ผลที่ควรได้

SALES READY = 43 rows

ส่วนที่ 4 — สาย Branch / Weather

เริ่มสายใหม่จาก *Branch_Master.csv*

ขั้นตอนที่ 11: อ่าน Branch Master

Node: CSV Reader

รับข้อมูลจาก: Branch_Master.csv

ต่อไปยัง: String Manipulation

วิธีตั้งค่า

1. เพิ่ม CSV Reader ตัวใหม่
2. เลือก Branch_Master.csv
3. ตรวจสอบว่ามี Branch_ID, Branch_Name, Latitude, Longitude
4. Execute

วัตถุประสงค์ของขั้นตอนนี้

Weather API ต้องใช้ Latitude และ Longitude เพื่อระบุตำแหน่งที่จะขอข้อมูลอากาศ

ผลที่ควรได้

3 rows

ส่วนที่ 5 — เรียก Weather API

ขั้นตอนที่ 12: สร้าง API URL

Node: String Manipulation

รับข้อมูลจาก: Branch Master

ต่อไปยัง: GET Request

วิธีตั้งค่า

1. Double-click String Manipulation
2. ที่ Expression ใส่:

```
join(
  "https://archive-api.open-meteo.com/v1/archive?latitude=",
  string($Latitude$),
  "&longitude=",
  string($Longitude$),
  "&start_date=2026-09-01&end_date=2026-09-15&daily=temperature_2m_mean,temperature_2m_max,precipitation_sum,weather_code&timezone=Asia%2FBangkok"
)
```

3. เลือก Append Column
4. ตั้งชื่อ Column ใหม่ว่า API_URL
5. Execute

วัตถุประสงค์ของขั้นตอนนี้

แต่ละ Branch มี Latitude/Longitude ต่างกัน จึงต้องสร้าง API URL คนละ URL

เราใช้ Branch Master 3 rows จึงเรียก API เพียง 3 ครั้ง ไม่ควรเอา Sales 43 rows มาเรียก API เพราะจะเรียกซ้ำโดยไม่จำเป็น

ผลที่ควรได้

3 rows และมี Column API_URL

บันทึกผล

จำนวน API URL 3

ขั้นตอนที่ 13: เรียก API

Node: GET Request

รับข้อมูลจาก: String Manipulation

ต่อไปยัง: JSON Path

วิธีตั้งค่า

1. เลือก Request URL จาก Column
2. เลือก Column = API_URL
3. Method = GET
4. Authentication = None
5. Execute

วัตถุประสงค์ของขั้นตอนนี้

Node นี้ส่ง HTTP GET Request ไปยัง Open-Meteo และรับ Weather JSON กลับมา

1 Branch = 1 Request ดังนั้น 3 Branch = 3 Requests

ตรวจผล

HTTP Status ควรเป็น 200

Response อยู่ใน body

ผลที่ควรได้

3 rows

ขั้นตอนที่ 14: ดึงข้อมูลที่ต้องการออกจาก JSON

Node: JSON Path

รับข้อมูลจาก: GET Request

ต่อไปยัง: Ungroup

วิธีตั้งค่า

Input JSON เลือก Response body

สร้าง Outputs:

Output	JSON Path	Type
Date	\$.daily.time[*]	String
Temperature_Mean	\$.daily.temperature_2m_mean[*]	Number
Temperature_Max	\$.daily.temperature_2m_max[*]	Number
Precipitation	\$.daily.precipitation_sum[*]	Number
Weather_Code	\$.daily.weather_code[*]	Integer

ให้ทุกตัวเป็น Collection / List

วัตถุประสงค์ของขั้นตอนนี้

API ส่งข้อมูลหลายวันอยู่ใน JSON เดียว จึงต้องเลือกข้อมูลเฉพาะส่วน daily ที่ต้องการก่อนนำมาแปลงเป็นตาราง

ผลที่ควรได้

ยังคง 3 rows แต่แต่ละ Weather Column เก็บข้อมูล 15 วันเป็น Collection

ขั้นตอนที่ 15: แยก Collection เป็นข้อมูลรายวัน

Node: Ungroup

รับข้อมูลจาก: JSON Path

ต่อไปยัง: String to Date&Time – Weather

วิธีตั้งค่า

เลือกพร้อมกัน:

Date
Temperature_Mean
Temperature_Max
Precipitation
Weather_Code

วัตถุประสงค์ของขั้นตอนนี้

ก่อน Ungroup: 1 Branch = 1 row แต่มีข้อมูล 15 วันอยู่ใน Collection

หลัง Ungroup: 1 Branch = 15 rows และ 1 row = 1 วัน

ผลที่ควรได้

3 Branch × 15 Days = 45 rows

บันทึกผล

จำนวนก่อน Ungroup 3

จำนวนหลัง Ungroup 45

จำนวนวันที่ต่อสาขา 15

ขั้นตอนที่ 16: แปลง Weather Date

Node: String to Date&Time

รับข้อมูลจาก: Ungroup

ต่อไปยัง: พักไว้เป็น WEATHER READY

วิธีตั้งค่า

1. เลือกเฉพาะ Date
2. Output type = Date
3. Input format = yyyy-MM-dd
4. Output = Replace
5. Execute

วัตถุประสงค์ของขั้นตอนนี้

เพื่อให้ Weather Date เป็นชนิดเดียวกับ Sales Date ก่อน Join

ผลที่ควรได้

WEATHER READY = 45 rows

ส่วนที่ 6 — รวมสาย Sales กับ Weather

ขั้นตอนที่ 17: Join Sales กับ Weather

Node: Joiner

รับข้อมูลจาก

Top / Left = SALES READY = 43 rows

Bottom / Right = WEATHER READY = 45 rows

Matching Criteria

กำหนด 2 เงื่อนไข:

Branch_ID = Branch_ID

Date = Date

Row Selection

เลือก:

- ✓ Matching rows
- ✓ Left unmatched rows

Output Columns จาก Weather

เลือก:

Branch_Name
Temperature_Mean
Temperature_Max
Precipitation
Weather_Code

ไม่ต้องเลือก Date และ Branch_ID ฝั่ง Weather ซ้ำ

วัตถุประสงค์ของขั้นตอนนี้

Weather ต้องตรงกับทั้งสาขาและวันที่ ถ้า Join ด้วย Date อย่างเดียว วันเดียวกันจะตรงกับ Weather ของหลายสาขา

ถ้า Join ด้วย Branch_ID อย่างเดียว Sales แต่ละรายการจะตรงกับ Weather หลายวัน จึงต้องใช้ Composite Key = Branch_ID + Date

ผลที่ควรได้

43 rows

ขั้นตอนที่ 18: ตรวจสอบว่า Weather มีข้อมูลหรือไม่

Node: Rule Engine

รับข้อมูลจาก: Final Joiner

ต่อไปยัง: Rain Status

สร้าง Column: Weather_Status

Rule:

```
MISSING $Temperature_Mean$ => "Missing"
TRUE => "Available"
```

วัตถุประสงค์ของขั้นตอนนี้

ถ้า Weather API ไม่มีข้อมูล เราไม่ควร Reject Sales เพราะ Sales ยังเป็น Transaction ที่ถูกต้อง จึงใช้ Flag บอกลักษณะแทน

ผลที่ควรได้

43 rows

ขั้นตอนที่ 19: สร้าง Rain Status

Node: Rule Engine

รับข้อมูลจาก: Weather Status

ต่อไปยัง: Temperature Level

สร้าง Column: Rain_Status

Rule:

```
MISSING $Precipitation$ => "Missing"
$Precipitation$ = 0 => "No Rain"
$Precipitation$ <= 10 => "Light Rain"
TRUE => "Heavy Rain"
```

วัตถุประสงค์ของขั้นตอนนี้

การแบ่งค่าฝนเป็นกลุ่มทำให้วิเคราะห์และสร้างกราฟใน Power BI ได้ง่ายกว่าการดูค่าปริมาณฝนดิบอย่างเดียว

ผลที่ควรได้

43 rows

ขั้นตอนที่ 20: สร้าง Temperature Level

Node: Rule Engine

รับข้อมูลจาก: Rain Status

ต่อไปยัง: Final Dataset

สร้าง Column: Temperature_Level

Rule:

```
MISSING $Temperature_Mean$ => "Missing"
```

```
$Temperature_Mean$ < 28 ==> "Cool"
$Temperature_Mean$ <= 31 ==> "Normal"
TRUE ==> "Hot"
```

วัตถุประสงค์ของขั้นตอนนี้

เพื่อสร้าง Feature เชิงกลุ่มสำหรับวิเคราะห์ยอดขายตามระดับอุณหภูมิ

ผลที่ควรได้

FINAL DATASET = 43 rows

ส่วนที่ 7 — สาย Reject: Quantity

กลับไป *Reject Output* จากขั้นตอนที่ 4

ขั้นตอนที่ 21: ใส่เหตุผล Reject ของ Quantity

Node: String Manipulation

รับข้อมูลจาก: FALSE Output ของ Quantity Splitter

ต่อไปยัง: Column Filter – Quantity Reject

วิธีตั้งค่า

Expression:

```
string("INVALID_QUANTITY")
```

เลือก Append Column

ชื่อ Column = Reject_Reason

วัตถุประสงค์ของขั้นตอนนี้

ถ้าเก็บ Reject อย่างเดียวแต่ไม่บอกเหตุผล ภายหลังจะไม่ทราบว่า Record ถูกปฏิเสธเพราะอะไร

ผลที่ควรได้

1 row

ขั้นตอนที่ 22: เลือก Columns ของ Quantity Reject

Node: Column Filter

รับข้อมูลจาก: String Manipulation – INVALID_QUANTITY

ต่อไปยัง: Concatenate

วิธีตั้งค่า

เลือกเก็บ:

```
Sales_ID
Date
Branch_ID
Product_ID
Quantity
Discount_Amount
Reject_Reason
```

วัตถุประสงค์ของขั้นตอนนี้

เราต้องทำ Schema ของ Reject แต่ละประเภทให้เหมือนกันก่อนนำไปรวม

ผลที่ควรได้

1 row

ส่วนที่ 8 — สาย Reject: Product

กลับไป *Reject Output* จากขั้นตอนที่ 7

ขั้นตอนที่ 23: ใส่เหตุผล Reject ของ Product

Node: String Manipulation

รับข้อมูลจาก: FALSE Output ของ Product Splitter

ต่อไปยัง: Column Filter – Product Reject

วิธีตั้งค่า

Expression:

```
string("PRODUCT_NOT_FOUND")
```

เลือก Append Column

ชื่อ Column = Reject_Reason

วัตถุประสงค์ของขั้นตอนนี้

ระบุให้ชัดว่า Record ถูก Reject เพราะ Product_ID ไม่พบใน Product Master

ผลที่ควรได้

1 row

ขั้นตอนที่ 24: เลือก Columns ของ Product Reject

Node: Column Filter

รับข้อมูลจาก: String Manipulation – PRODUCT_NOT_FOUND

ต่อไปยัง: Concatenate

วิธีตั้งค่า

เลือกเก็บเหมือน Quantity Reject:

Sales_ID

Date

Branch_ID

Product_ID

Quantity

Discount_Amount

Reject_Reason

วัตถุประสงค์ของขั้นตอนนี้

เพื่อให้สองตารางสามารถ Concatenate กันได้ง่ายและมี Schema เดียวกัน

ผลที่ควรได้

1 row

ส่วนที่ 9 — รวม Reject Log

ขั้นตอนที่ 25: รวม Reject

Node: Concatenate

รับข้อมูลจาก:

Input 1 = Quantity Reject

Input 2 = Product Reject

ต่อไปยัง: DB Writer – reject_log

วิธีตั้งค่า

ต่อ Output ของ Column Filter ทั้งสองสายเข้ากับ Concatenate แล้ว Execute

วัตถุประสงค์ของขั้นตอนนี้

เราไม่ได้ต้องการ Join ตาม Key แต่ต้องการนำ Record จากสองตารางมาต่อกันในแนวตั้ง

ผลที่ควรได้

Reject Log = 2 rows

ส่วนที่ 10 — สาย SQLite Connection

เริ่มสายใหม่

ขั้นตอนที่ 26: สร้าง SQLite Connection

Node: SQLite Connector

รับข้อมูลจาก: ไม่มี เป็น Node สำหรับ Connection

ต่อไปยัง: DB Writer, DB Table Selector

วิธีตั้งค่า

1. เพิ่ม SQLite Connector
2. เปิด Configure
3. Location = Path
4. เลือกโฟลเดอร์สำหรับเก็บ Database
5. ตั้งชื่อไฟล์ week3_sales.sqlite
6. Database Dialect = SQLite ถ้ามีตัวเลือก
7. ไม่ต้องกรอก Host, Port, Username, Password
8. Execute

วัตถุประสงค์ของขั้นตอนนี้

SQLite เป็นฐานข้อมูลแบบ File-based ไม่ต้องมี Database Server

ผลที่ควรได้

Node Execute สำเร็จและได้ Database Connection

ส่วนที่ 11 — Load ข้อมูลเข้าสู่ SQLite

ขั้นตอนที่ 27: เขียน Final Dataset

Node: DB Writer

รับข้อมูลจาก:

- Data Table = Final Dataset 43 rows
- DB Connection = SQLite Connector

ต่อไปยัง: SQLite table sales_weather

วิธีตั้งค่า

1. ต่อ Final Dataset เข้าพอร์ต Data Table ของ DB Writer
2. ต่อ SQLite Connector เข้าพอร์ต Database Connection
3. ตั้ง Table name = sales_weather
4. ในการทดลองครั้งแรกใช้ Create/Overwrite/Replace existing table ตามชื่อที่ KNIME version แสดง
5. หลีกเลี่ยง Append เมื่อต้อง Execute ซ้ำ เพราะจะทำให้ข้อมูลซ้ำ
6. Execute

วัตถุประสงค์ของขั้นตอนนี้

นี่คือขั้นตอน Load ของ ETL

ผลที่ควรได้

sales_weather = 43 rows

ขั้นตอนที่ 28: เขียน Reject Log

Node: DB Writer

รับข้อมูลจาก:

- Data Table = Reject Log 2 rows
- DB Connection = SQLite Connector

วิธีตั้งค่า

1. เพิ่ม DB Writer อีกตัว
2. ดึง Reject Log เข้าพอร์ต Data Table
3. ดึง SQLite Connector เข้าพอร์ต Database Connection
4. ตั้ง Table = reject_log
5. Execute

วัตถุประสงค์ของขั้นตอนนี้

Reject Data ควรถูกเก็บไว้เพื่อ Audit และตรวจสอบคุณภาพข้อมูลภายหลัง ไม่ควรถูกลบทิ้งโดยไม่มีหลักฐาน

ผลที่ควรได้

reject_log = 2 rows

ขั้นตอนที่ 29: อ่านข้อมูลจาก SQLite กลับมาตรวจ

Nodes: DB Query Reader

วิธีตั้งค่า

1. เพิ่ม Node DB Query Reader
2. ดึง Output ของ SQLite Connector เข้ากับ DB Query Reader
3. Double-click ที่ DB Query Reader
4. ในช่อง SQL Statement ใส่

```
SELECT *
FROM sales_weather;
```
5. กด Apply / OK
6. Execute DB Query Reader
7. เปิด Output Table เพื่อตรวจสอบข้อมูล

วัตถุประสงค์ของขั้นตอนนี้

เพื่ออ่านข้อมูลที่ถูกบันทึกไว้ใน SQLite กลับมาตรวจสอบ และยืนยันว่าขั้นตอน Load ทำงานสำเร็จ ข้อมูลที่อยู่ในฐานข้อมูลมีจำนวนแถวและค่าตรงกับ Final Dataset ก่อนบันทึกผลที่ควรได้

Final Dataset ก่อน Load = 43 rows

ข้อมูลที่อ่านกลับจาก sales_weather = 43 rows

ส่วนที่ 12 — ส่งออกสำหรับ Dashboard

ขั้นตอนที่ 30: ส่งออก Final Dataset

Node: CSV Writer

รับข้อมูลจาก: Final Dataset

ต่อไปยัง: Final_Sales_Weather.csv

วิธีตั้งค่า

1. เพิ่ม CSV Writer
2. ต่อ Final Dataset เข้ากับ CSV Writer
3. กำหนดชื่อไฟล์ Final_Sales_Weather.csv
4. Execute

วัตถุประสงค์ของขั้นตอนนี้

SQLite ใช้เป็น Storage สำหรับฝึก Load ส่วน CSV ใช้เป็น Dataset สำหรับ Power BI โดยไม่ต้องติดตั้ง SQLite ODBC เพิ่ม

ผลที่ควรได้

43 rows

สิ่งที่ต้องส่ง

1. ภาพ Workflow ทั้งหมด
2. Final_Sales_Weather.csv
3. week3_sales.sqlite
4. Screenshot ตาราง sales_weather จาก DB Reader
5. Power BI Dashboard
6. คำตอบคำถามสะท้อนคิด
7. ให้หาคำสั่ง SQL จำนวน 5 ข้อ โดยแต่ละข้อประกอบด้วย คำสั่ง SQL ที่เขียนเอง + Screenshot ของ SQL ใน DB Query Reader + Screenshot ตารางผลลัพธ์หลัง Execute + คำตอบหรือข้อสรุปจากผลลัพธ์

The screenshot displays the KNIME Analytics Platform interface. The top bar shows the 'lab3_1' workspace. The main area features a workflow diagram with nodes for 'DB Writer', 'DB Query Reader', and 'CSV Writer'. A 'DB Query Reader' node is highlighted, showing a SQL query in the right-hand pane:

```
1 SELECT
2   COUNT(Sales_ID) AS Total_Transactions,
3   SUM(Net_Sales) AS Total_Net_Sales,
4   AVG(Net_Sales) AS Avg_Net_Sales
5 FROM sales_weather;
```

Below the workflow, the '1: KNIME data table' is displayed, showing the results of the query:

#	RowID	Total_Transactions	Total_Net_Sales	Avg_Net_Sales
1	Row0	43	18,005	418.721

KNIME Analytics Platform

Home lab3_1 + Preferences Menu natakornkhawyal

Save Execute Cancel Reset Hub - lab3_1 Deploy on Hub

Info Nodes Explorer K-AI Monitor

DB Query Reader

```
1 SELECT
2   Category,
3   SUM(Quantity) AS Total_Quantity,
4   SUM(Net_Sales) AS Total_Net_Sales
5 FROM sales_weather
6 GROUP BY Category
7 ORDER BY Total_Net_Sales DESC;
```

Discard Apply Apply and Execute

1: KNIME data table Flow Variables

Rows: 3 | Columns: 3 | Sorting Filter

#	RowID	Category	Total_Quantity	Total_Net_Sales
1	Row0	Drink	184	8,360
2	Row1	Snack	76	5,140
3	Row2	Bakery	74	4,505

Search

15:52 6/10/2569

KNIME Analytics Platform

Home lab3_1 + Preferences Menu natakornkhawyal

Save Execute Cancel Reset Hub - lab3_1 Deploy on Hub

Info Nodes Explorer K-AI Monitor

DB Query Reader

```
1 SELECT
2   Branch_ID,
3   COUNT(Sales_ID) AS Order_Count,
4   SUM(Gross_Sales) AS Total_Gross,
5   SUM(Net_Sales) AS Total_Net
6 FROM sales_weather
7 GROUP BY Branch_ID
8 ORDER BY Total_Net DESC;
```

Discard Apply Apply and Execute

1: KNIME data table Flow Variables

Rows: 3 | Columns: 4 | Sorting Filter

#	RowID	Branch_ID	Order_Count	Total_Gross	Total_Net
1	Row0	B01	15	6,335	6,285
2	Row1	B03	14	5,995	5,950
3	Row2	B02	14	5,820	5,770

Search

15:52 6/10/2569

KNIME Analytics Platform

Home lab3_1 + Preferences Menu natakornkhawai

Save Execute Cancel Reset Hub - lab3_1 Deploy on Hub

Info Nodes Explorer K-AI Monitor

DB Query Reader

```
1 SELECT
2   Reject_Reason,
3   COUNT(*) AS Rejected_Count
4 FROM reject_log
5 GROUP BY Reject_Reason;
```

Discard Apply Apply and Execute

► 1: KNIME data table Flow Variables

Rows: 1 | Columns: 2 Sorting Filter Table Statistics Search

#	RowID	Reject_Reason	Rejected_Count
1	Row0	INVALID_QUANTITY	2

Windows Search 64% 15:53 6/10/2569

KNIME Analytics Platform

Home lab3_1 + Preferences Menu natakornkhawai

Save Execute Cancel Reset Hub - lab3_1 Deploy on Hub

Info Nodes Explorer K-AI Monitor

DB Query Reader

```
1 SELECT
2   Date,
3   Branch_ID,
4   Precipitation,
5   SUM(Net_Sales) AS Daily_Net_Sales
6 FROM sales_weather
7 WHERE Precipitation > 10
8 GROUP BY Date, Branch_ID
9 ORDER BY Precipitation DESC;
```

Discard Apply Apply and Execute

► 1: KNIME data table Flow Variables

Rows: 13 | Columns: 4 Sorting Filter Table Statistics Search

#	RowID	Date	Branch_ID	Precipitation	Daily_Net_Sales
1	Row0	2026-09-12	B02	20.8	390
2	Row1	2026-09-10	B03	17.9	375
3	Row2	2026-09-12	B01	16.4	345
4	Row3	2026-09-08	B02	14.8	405
5	Row4	2026-09-10	B02	14.8	595
6	Row5	2026-09-11	B01	14.6	550

Windows Search 64% 15:53 6/10/2569